

Exercício Programa 3

MAC0422 - Sistemas Operacionais (2026)

Simulador de Algoritmos de Substituição de Páginas



Apresentação de Slides

Roteiro

3 - Funcionamento dos Algoritmos

4 - Automação dos Testes

5 - Gráfico Comparativo

6 a 8 - Sobre Traces Utilizados

9 - Configurações Utilizadas

10 - Referências

Funcionamento dos Algoritmos

Estruturas de dados baseadas nessa organização: a **memória virtual** contém suas entradas com bits R e M, bit presente/ausente, e endereço no quadro de páginas; a **memória física** contém suas entradas com o PID do processo responsável, e o endereço correspondente na tabela de páginas; a **MMU** controla quantidades de acessos, hits, faults, além de fila e matriz, usadas a depender do algoritmo de paginação.

NRU { - Começa a busca em ponto aleatório da *tabela de páginas*.

- Itera sobre aquelas páginas presentes na memória física, da classe 0 à classe 3 definidas pelos bit R e M, até encontrar a vítima.

Chance { - Inicializa a fila com o *quadro de páginas*, se necessário.

- Desenfileira. Se o bit R == 1, enfileira de volta a página com bit R == 0, e repete a busca.

LRU { - Varre toda a matriz LRU em busca da linha com menor valor binário do contador de R, quando a entrada da *tabela de páginas* está de fato presente na memória física.

- O índice da vítima é atualizado se encontrado alguém menor, considerando m bits mais significativos.

Em todos eles : após encontrar o endereço da vítima, atualiza as estruturas de dados nas duas memórias (para a página sainte da virtual, e a página nova na física) e retorna ponteiro para o quadro correto na memória física. A atualização da página entrante fica a cargo da função chamadora desses algoritmos, “page_fault_handler”.

Automação dos Testes

```
#!/bin/bash
(...)

echo "iniciando fase de testes "

for i in {1..3}; do
    for j in {1..3}; do
        case "${i}" in
            1)
                ./ep3 ep3-memfisica-exemplo01.pgm ep3-memvirtual-exemplo01.pgm
                ${j} < trace-nru > trace-nru-${j}.log
                ;;
            2)
                ./ep3 ep3-memfisica-exemplo01.pgm ep3-memvirtual-exemplo01.pgm
                ${j} < trace-chance > trace-chance-${j}.log
                ;;
            3)
                ./ep3 ep3-memfisica-exemplo01.pgm ep3-memvirtual-exemplo01.pgm
                ${j} < trace-lru > trace-lru-${j}.log
                ;;
            *)
                ;;
        esac
        cp ../ep3-memfisica-exemplo01.pgm ../ep3-memvirtual-exemplo01.pgm .
    done
done
```



As 9 execuções levam um tempo na ordem de apenas 100 milissegundos para serem completadas. Todos com 650 acessos.

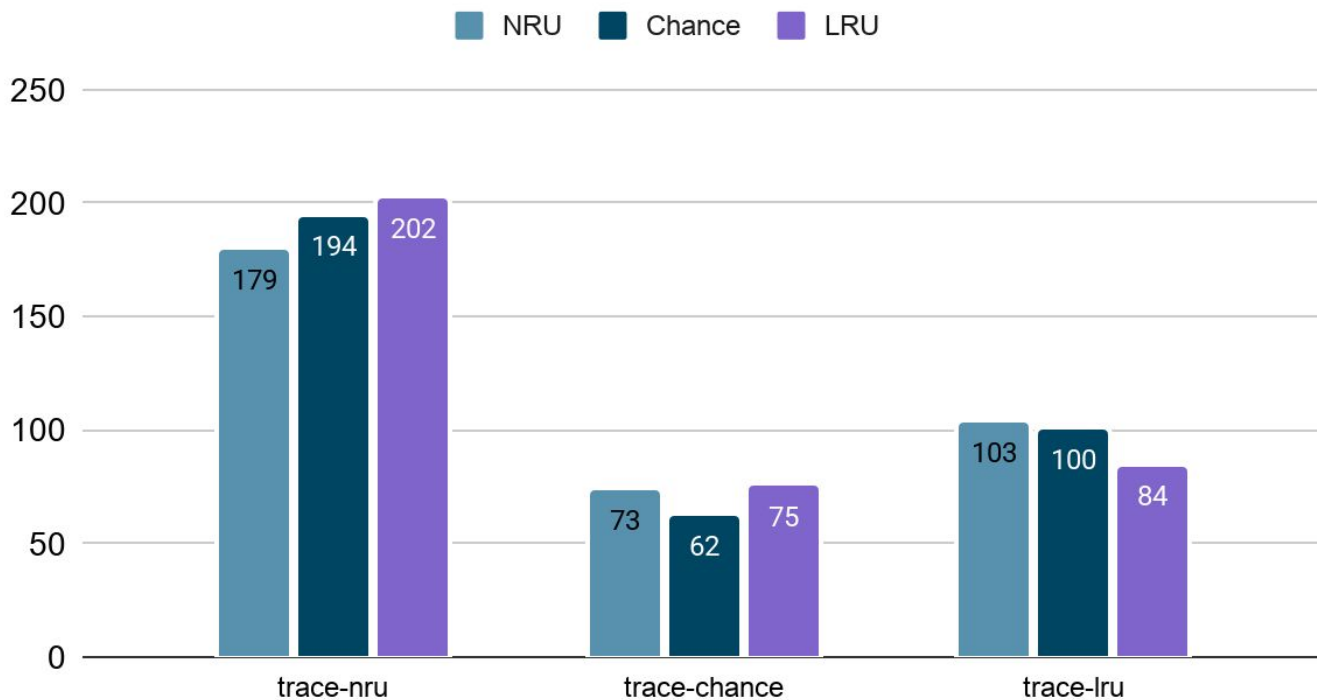
Em 30 tentativas de execução, os resultados nas quantidades de falhas de página foram idênticos, mesmo que o algoritmo NRU insira alguma aleatoriedade na substituição – provavelmente não houveram empates suficientes nos traces usados e gerou esse comportamento determinístico.

Os resultados de um teste são salvos em um log com as linhas da saída contendo “> hit” ou “> fault”, e a contabilização é feita usando o recurso de encontrar da própria IDÉ (VSCode).

Vale notar que os pgms usados foram apenas os fornecidos na página da disciplina: “ep3-memfisica-exemplo01.pgm” e “ep3-memvirtual-exemplo01.pgm”, com a restauração dos mesmos usando as cópias num diretório pai após cada teste.

Comparação do Desempenho (por trace)

Quantidade de Page Faults



Traces (650 acessos)

NRU - ciclos de 25 acessos

PID 0	PID 64	PID 128	PID 192
- Leituras > Escritas - Usa 16 das 16 páginas - Aparece nos ciclos de acessos em que 192 não	- Leituras > Escritas - Usa 14 das 16 páginas - Aparece em todos ciclos de acessos	- Leituras < Escritas - Usa 11 das 16 páginas - Aparece em todos ciclos de acessos	- Leituras < Escritas - Usa 10 das 16 páginas - Aparece em só 10 dos 26 ciclos de acessos

Chance - ciclos de 28 acessos, 7 p/ cada processo

PID 0	PID 64	PID 128	PID 192
- Leituras = Escritas - Usa 9 das 16 páginas - Algumas usadas no início voltam a ser usadas depois	- Leituras > Escritas - Usa 12 das 16 páginas - Algumas usadas no início voltam a ser usadas depois, junto àquelas sempre usadas. - Passa a andar mais lentamente.	- Leituras < Escritas - Usa 16 das 16 páginas - Passa a exigir cada vez mais páginas sendo recorrentemente acessadas. - Anda lentamente.	- Leituras = Escritas - Usa 16 das 16 páginas - Sempre usa todas as páginas em todos ciclos - Passa a andar mais rapidamente nos últimos ~8 ciclos

LRU - ciclos de 10 a 13 acessos, todas páginas usadas

PID 0	PID 64	PID 128	PID 192
- Leituras = Escritas - Sempre lê numa posição e escreve na página seguinte - Anda lentamente a partir de uma base	- Leituras = Escritas - Parecido com o PID 0 mas faz um pouco mais de leitura do meio pro final	- Leituras > Escritas - Anda lentamente a partir de uma base, mas começa a acessar endereços em intervalos maiores com o tempo (mais rápido)	- Leituras > Escritas - Anda rapidamente a partir de uma base, e começa a acessar endereços em intervalos maiores com o tempo (mais rápido)

Traces

- ❑ Gerados de forma parcialmente automática (assistida) via loops com auxílio de um bash script para criar alguns padrões nos acessos. Alguns trechos ilustrando isso:

PID 192
com
acessos
periódicos
em
trace-nru

```
read -p "Selecione 0 para processo 192 ausente, 1 para presente: " valor
if [ "${valor}" -eq "0" ]; then
    echo "64 L $(( (16000+i*9000) % 56000 ))" >> trace-nru
    echo "64 L $(( (18000+i*9000) % 56000 ))" >> trace-nru
    echo "64 L $(( (20000+i*9000) % 56000 ))" >> trace-nru
    echo "64 L $(( (22000+i*9000) % 56000 ))" >> trace-nru
    echo "64 E 0" >> trace-nru
    echo "64 E 5000" >> trace-nru
    echo "0 L $(( (i*1000) % 65536 ))" >> trace-nru
```

```
else
    echo "64 L $(( (18000+i*9000) % 56000 ))" >> trace-nru
    echo "64 L $(( (20000+i*9000) % 56000 ))" >> trace-nru
    echo "64 L $(( (22000+i*9000) % 56000 ))" >> trace-nru
    echo "64 E 0" >> trace-nru
    echo "192 L 65000" >> trace-nru
    echo "192 E 1000" >> trace-nru
    echo "192 E 5000" >> trace-nru
    echo "192 E 9000" >> trace-nru
    echo "192 E 13000" >> trace-nru
```

```
echo "192 L $(( (3000*i) % 65536 ))" >> trace-chance
echo "192 L $(( (1000+3000*i) % 65536 ))" >> trace-chance
echo "192 E $(( (2000+3000*i) % 65536 ))" >> trace-chance
echo "192 E $(( (3000+3000*i) % 65536 ))" >> trace-chance
echo "192 L $(( (4000+3000*i) % 65536 ))" >> trace-chance
echo "192 E $(( (5000+3000*i) % 65536 ))" >> trace-chance
echo "192 E $(( (6000+3000*i) % 65536 ))" >> trace-chance
echo "64 L 0" >> trace-chance
echo "64 L $(( (5000+5000*i) % 44000 ))" >> trace-chance
echo "64 L $(( (9000+5000*i) % 44000 ))" >> trace-chance
echo "64 L $(( (15000+5000*i) % 44000 ))" >> trace-chance
echo "64 L $(( (20000+5000*i) % 44000 ))" >> trace-chance
echo "64 L $(( (25000+5000*i) % 44000 ))" >> trace-chance
echo "64 E 50000" >> trace-chance
```

Tempos de
CPU iguais
em
trace-chance

```
for i in {1..60}; do
    base128=$(( 2000 * i ))
    base192=$(( 3000 * i ))
    base64=$(( 1500 * i ))
    base0=$(( 1000 * i ))

    #
    addr() {
        echo $(( $1 % limit ))
    }

    #
    if (( i < 8 )); then
        echo "128 L $(addr $((base128)))" >> trace-lru
        echo "128 E $(addr $((base128+4000)))" >> trace-lru
        echo "128 L $(addr $((base128+8000)))" >> trace-lru

        #
        echo "192 L $(addr $((base192)))" >> trace-lru
        echo "192 L $(addr $((base192+3000)))" >> trace-lru
        echo "192 E $(addr $((base192+6000)))" >> trace-lru

        #
        echo "64 L $(addr $((base64)))" >> trace-lru
        echo "64 E $(addr $((base64+2000)))" >> trace-lru

        #
        echo "0 L $(addr $((base0)))" >> trace-lru
        echo "0 E $(addr $((base0+4000)))" >> trace-lru

        #
    elif (( i < 16 )); then
        echo "128 L $(addr $((base128)))" >> trace-lru
        echo "128 L $(addr $((base128+2000)))" >> trace-lru
        echo "128 E $(addr $((base128+6000)))" >> trace-lru
        echo "128 L $(addr $((base128+10000)))" >> trace-lru
```

Bases móveis que favorecem
a progressão dos blocos de
acessos no espaço de
endereços, em trace-lru

Traces

NRU

- ❑ Ao ter mais escritas que os outros, favorece uma melhor separação de classes
- ❑ Como páginas que aparecem inicialmente voltam a ser importantes depois, é menos prejudicado com o envelhecimento
- ❑ *Podemos imaginar:* processos cíclicos onde ocorre leitura em loops; o PID 192 seria um I/O bound que escreve com frequência em buffers; escalonador por prioridades (dinâmica para I/O bound) como escalonador principal

Chance

- ❑ “Meio termo” com relação aos outros, ficando mais perto do NRU no gráfico, mas capturando envelhecimento e localidade temporal melhor que ele
- ❑ Aparente menor quantidade de page faults após zeragem dos bit R (acessos múltiplos de 100), possivelmente por ter a estrutura de decisão (fila) que favorece uma adaptação mais rápida já que páginas reutilizadas podem voltar a ser marcadas antes de serem substituídas
- ❑ *Podemos imaginar:* os 4 como processos CPU bound consumindo longos tempos de CPU; cálculos constantes e leitura sequencial de funções; Round-Robin como escalonador principal.

LRU

- ❑ Páginas lidas no passado se tornam menos importantes depois com o estabelecimento de “bases”, e assim se lê a partir de determinados pontos do código. Melhor a lidar com envelhecimento
- ❑ Reduz o número de page faults em momentos onde outros algoritmos ainda sofrem com decisões baseadas em histórico menos preciso.
- ❑ *Podemos imaginar:* processos CPU bound numa inicialização de sistema com diferentes fases de execução; alguns vão se tornando mais importantes que outros, cujas tarefas ficam em background; escalonador de prioridades toma essas decisões

Configurações Utilizadas e Conclusões



Intel Core i5-13420H (**12 cores**) 13ª gen,
Fedora Linux 43 (Workstation Edition),
(12 núcleos lógicos, 8 físicos)

- ❑ Podemos imaginar diferentes cenários onde cada algoritmo de paginação gera menos falhas de página. Cada um tem suas vantagens e desvantagens
- ❑ NRU tem como vantagem não precisar de mais estruturas auxiliares, mas pode ser simples demais
- ❑ Segunda Chance evita que se jogue fora páginas importantes – o que um FIFO faria – mas pode sofrer se houver cargas de trabalho de processos mais variadas
- ❑ LRU aproxima melhor de um algoritmo ótimo, mas é mais complexo e custoso para as atualizações necessárias a cada acesso

Referências

Livros:

Andrew S. Tanenbaum, "Sistemas operacionais modernos", Pearson Education do Brasil Pearson, 2010.

Manpages e Docs:

<https://man7.org/linux/man-pages/man3/sscanf.3.html>

https://linux.die.net/man/3/strtok_r